Validation Plan: Harmonic Oscillator Path Integral with MCMC

Javad Yousefian Alexandra VonHaven Kevin Geumhan

Version 1, November 9, 2026

1 Introduction

In this project, we plan to sample the path integral of the quantum harmonic oscillator in imaginary time using Markov chain Monte Carlo methods. We will write our own Metropolis sampler and also use `emcee` [1] so we can directly compare the two methods. This document describes our code validation plan and was written before any code, so it does not include results.

Our main goal is to check if our Metropolis code samples the correct distribution. The harmonic oscillator is a good choice for validation because its lattice distribution is Gaussian, which lets us compare with exact analytical results.

2 The model

The Euclidean action of the oscillator is

$$S_E = \int_0^{T_\tau} d\tau \left[\frac{m}{2}\dot{x}^2 + \frac{m\omega^2}{2}x^2 \right]. \tag{1}$$

We divide the time into N steps of size ϵ and use this discretization and rescaling:

$$a = \omega\epsilon, \qquad \kappa = \frac{a^2}{4}, \qquad g = \frac{1-\kappa}{1+\kappa}, \qquad x_j = \sqrt{\frac{\hbar\epsilon}{m(1+\kappa)}}\, u_j. \tag{2}$$

The endpoints are fixed, $u_0 = u_N = 0$, so the free variables are $u_1, \ldots, u_{N-1}$. This is $D = N - 1$ dimensions ($D = 63$ for our main lattice). The action becomes

$$\mathcal{S}(u) = \sum_{j=1}^{N-1} u_j^2 - g \sum_{j=0}^{N-1} u_j u_{j+1}, \tag{3}$$

and we sample $\pi(u) \propto e^{-\mathcal{S}(u)}$. We keep the endpoints fixed and do not use periodic boundaries, because that would change the distribution.

Exact answer. The action can be written as $\mathcal{S} = \frac{1}{2}u^T Q u$, where Q has 2 on the diagonal and $-g$ next to the diagonal. Since $|g| < 1$, Q is positive definite, so π is a Gaussian with mean zero and covariance Q^{-1}:

$$\langle u_j \rangle = 0, \qquad \langle u_j u_k \rangle = (Q^{-1})_{jk}, \qquad \langle u_j^4 \rangle = 3\left[(Q^{-1})_{jj}\right]^2. \tag{4}$$

We can get Q^{-1} from NumPy, but there is also a closed formula for it. With $\mu = 2\,\mathrm{artanh}(a/2)$,

$$(Q^{-1})_{jk} = \frac{\sinh(\mu \min(j,k))\,\sinh(\mu(N - \max(j,k)))}{g \sinh\mu\,\sinh(\mu N)}. \tag{5}$$

Far from the ends this is about $\frac{1+\kappa}{2a}\rho^{|j-k|}$ with $\rho = (2-a)/(2+a)$. For $a = 0.25$ this gives $\langle u_j^2\rangle \approx 2.03$, $\langle u_j u_{j+1}\rangle \approx 1.58$ and $\langle u_j u_{j+2}\rangle \approx 1.23$. Near the ends the variance is smaller: for $a = 0.25$ it is 0.802 at $j = 1$, 1.288 at $j = 2$ and 1.759 at $j = 4$.

Symmetries. The distribution does not change if we flip the sign of the path ($u \to -u$) or if we reverse it in time ($u_j \to u_{N-j}$). So $\langle u_j\rangle = 0$ and $\langle u_j u_k\rangle = \langle u_{N-j} u_{N-k}\rangle$. There are no degeneracies, because Q is positive definite and there is only one peak, at $u = 0$.

Energy gap. In the middle of the lattice the correlation falls like ρ^k, so the decay rate per unit time is

$$\frac{\omega_{\mathrm{lat}}}{\omega} = \frac{2}{a}\,\mathrm{artanh}\Big(\frac{a}{2}\Big). \tag{6}$$

For small a this goes to 1, which is the continuum gap $E_1 - E_0 = \hbar\omega$. At $a = 1$ it is 1.099.

3 What the code will do

1. Make a path with $u_0 = u_N = 0$ and compute its action, Eq. (3).
2. Sample the path with our Metropolis code, and separately with `emcee`.
3. Measure $\langle u_j\rangle$, $\langle u_j^2\rangle$, $\langle u_j^4\rangle$ and the correlation $\langle u_{j_0} u_{j_0+k}\rangle$, where $j_0 = N/2$ is the middle site.
4. Fit the decay of the correlation to get the energy gap.
5. Compute error bars and convergence checks.
6. Save the settings, the seed, the results and the plots, so every run can be repeated.

4 Samplers

Our Metropolis sampler. In one sweep we go through all sites $j = 1, \ldots, N-1$. For each site we propose $u_j' = u_j + \delta$, with δ uniform in $[-\Delta, \Delta]$. The change of the action only depends on the two neighbours:

$$\Delta\mathcal{S} = \delta\,[\delta + 2u_j - g(u_{j-1} + u_{j+1})]. \tag{7}$$

We accept with probability $\min(1, e^{-\Delta\mathcal{S}})$. If we reject, we keep the old value and still count it as the next sample. To make it faster, we update all odd sites at the same time and then all even sites. This is allowed because odd sites are not neighbours of each other.

The expected acceptance rate can also be calculated exactly. Given its neighbours, u_j is Gaussian with variance 1/2 (for any a and N), and averaging over this gives

$$A(\Delta) = \mathrm{erfc}\left(\frac{\Delta}{2}\right) + \frac{2}{\sqrt{\pi}\,\Delta}\left(1 - e^{-\Delta^2/4}\right). \tag{8}$$

For example $A(1) = 0.729$, $A(2) = 0.514$ and $A(3) = 0.370$. It does not depend on a, so the acceptance should be the same for every lattice spacing.

We will try $\Delta = 0.5$, 1, 1.5, 2, 2.5, 3, and 4 in short pilot runs. After that, we will choose the value that gives the most effective samples with an acceptance rate between about 35% and 65%. We expect this value to be around 2.5.

emcee. For the second sampler, we use `emcee.EnsembleSampler` with the same log probability $-\mathcal{S}(u)$ and at least $2D$ walkers. Because emcee works differently from our sampler, we expect the averages to match but not the individual chains. emcee is slow in high dimensions, so we will mainly use it for $N = 64$. If it is too slow, we will switch to $N = 16$ and note this change in the report. We will run emcee four times independently so we can compute $\hat{R}$.

Settings. Our main lattice is $N = 64$ and $a = 0.25$, so $T = Na = 16$. For the energy gap test, we also use $N = 16$ with $a = 1$, and $N = 32$ with $a = 0.5$. We start four chains, each with 8,000 sweeps of burn-in and 40,000 sweeps for sampling. If the convergence checks fail, we will run longer. Two chains start at $u = 0$, and the other two start from a random path that is much wider than the real distribution.

5 Error bars and when a test passes

Our samples are correlated, so we cannot use the normal $\sigma/\sqrt{M}$ error. We compute the integrated autocorrelation time

$$\tau_{\text{int}} = \frac{1}{2} + \sum_{\ell=1}^{\ell_{\max}} \rho(\ell), \tag{9}$$

where $\rho(\ell)$ is the autocorrelation at lag ℓ and $\ell_{\max}$ is chosen with Sokal's window [3]. The effective sample size is then $M/(2\tau_{\text{int}})$ for M samples. emcee uses $1 + 2\sum\rho$, which is $2\tau_{\text{int}}$, so we divide emcee's number by 2. For the error bars we use batch means, with batches at least $20\tau_{\text{int}}$ long.

For the statistical tests we compare with the exact value using

$$z = \frac{|\text{measured} - \text{exact}|}{\text{error bar}}, \tag{10}$$

and a test passes if $z \le 3.5$. We do many comparisons, so one of them could fail just by chance. If that happens, we run the test again with a different seed before we decide that there is a bug, and we report both runs. We will not change these limits after we see the results.

6 Tests

For every test we write what we check, when it passes, and what it would mean if it fails.

Test 1: Action and the update

We compare $\Delta\mathcal{S}$ from Eq. (7) with the difference of the full action for 1000 random paths. They should agree to 10^{-10}. We also check that $\frac{1}{2}u^T Q u$ gives the same action, that the acceptance probability is always between 0 and 1, and that the fast odd/even sweep gives exactly the same result as a simple loop over the sites when we give both the same random numbers. If this fails, there is probably a sign or index mistake, or a problem at the endpoints.

Test 2: Exact covariance

We check that our Q is symmetric and positive definite, that its eigenvalues are $2 - 2g\cos(r\pi/N)$ for $r = 1, \ldots, N-1$, and that the closed formula Eq. (5) agrees with NumPy's inverse to 10^{-10}. For $a = 0.25$ the numbers should match the values in Section 2. All the other tests compare with this, so if it fails we fix it first.

Test 3: Error bar code

Before we trust our error bars, we test the autocorrelation and batch-means code on AR(1) data, $x_t = \phi x_{t-1} + \sqrt{1-\phi^2}\,\epsilon_t$, where the exact answer is $\tau_{\mathrm{int}} = (1+\phi)/(2(1-\phi))$. We use $\phi = 0.5$, 0.9 and 0.99. The estimate of τ_{int} should be within 10%, the batch-means error within 20%, and emcee's value should be about $2\tau_{\mathrm{int}}$. We also check that $\hat{R}$ is below 1.01 for four normal chains and above 1.01 if we shift one of them. If this fails, none of our error bars can be trusted.

Test 4: Very small lattices

For $N = 2$ there is only one variable, and it should be Gaussian with variance $1/2$. For $N = 3$ there are two variables, and $u_1 + u_2$ and $u_1 - u_2$ should be independent Gaussians with variances $2/(2-g)$ and $2/(2+g)$. We run 200,000 sweeps and do a Kolmogorov–Smirnov test against these Gaussians, which passes if $p > 0.001$. The variance, the fourth moment and the acceptance rate should also pass $z \le 3.5$. This test can find mistakes that Test 1 cannot, for example if we forget to save the old value when a step is rejected.

Test 5: Known answer at $N = 64$

This is the main test. With $N = 64$ and $a = 0.25$ we measure $\langle u_{j_0} \rangle$, $\langle u_{j_0} u_{j_0+k} \rangle$ for $k = 0, 1, 2, 4, 8$, $\langle u_{j_0}^4 \rangle$, and the variance at the sites near the ends ($j = 1, 2, 4$ and $N-4$, $N-2$, $N-1$). All of them should pass $z \le 3.5$ against Eq. (4). If something fails, the sampler is biased, there is a bug near the ends, or our error bars are wrong.

Test 6: Symmetries and starting points

Because of the symmetries, $\langle u_j \rangle$ should be zero at $j = N/4$, $N/2$ and $3N/4$, and $\langle u_{j_0} u_{j_0+4} \rangle$ should be equal to $\langle u_{j_0} u_{j_0-4} \rangle$. We also compare the chains that started at $u = 0$ with the ones that started from a wide random path, and they should agree within 3 error bars. If not, the burn-in is too short, the chains do not mix well, or a bug breaks one of the symmetries.

Test 7: Convergence

We look at trace plots of u_{j_0}, $u_{j_0}^2$ and the path average $\bar{u}$. The path average changes the slowest, so it is the most important one. For all the numbers we report, we need $\hat{R} < 1.01$ (the rank-normalized split $\hat{R}$ from [2]) and an effective sample size above 400 in total. If this fails, we run longer or change Δ.

Test 8: Acceptance rate

For each Δ in the pilot runs, the measured acceptance should agree with $A(\Delta)$ from Eq. (8) ($z \le 3.5$), both for $a = 0.25$ and for $a = 1$. Since A does not depend on a, both should give the same numbers.

If it fails, there is a mistake in the δ^2 part of $\Delta\mathcal{S}$, in the proposal width, or in the accept/reject step.

Test 9: Our sampler against emcee

We compare $\langle u_{j_0} \rangle$, $\langle u_{j_0}^2 \rangle$ and the correlation at $k = 1, 4, 8$ from our sampler and from emcee at $N = 64$. The difference divided by the combined error bar should be at most 3. Both samplers should also agree with the exact answer, because two samplers can agree with each other and still both be wrong. If one of them does not converge, we do not use it to judge the other one.

Test 10: Getting back the energy gap

This is our injection test: we put ω into the code (through a), and we check that we get it back from the samples. For $a = 1$, 0.5 and 0.25 (all with $T = 16$) we fit $\langle u_{j_0} u_{j_0+k} \rangle \propto e^{-\hat{\mu} k}$ for times ka between 0.5 and 3, using the middle part of the lattice. Then $\hat{\mu}/a$ should agree with Eq. (6) within 3.5 error bars, and the fit should be good ($p > 0.001$). At $a = 1$ the lattice value 1.099 is about 10% above the continuum value, so we hope to see this difference. If the test fails, the fit is wrong or the sampler is biased at long distances.

7 Test execution

When we write the code, we will turn these tests into `pytest` files where possible, so they can run automatically. Tests 1–4 and 8 are fast and will run on every pull request. Tests 5–7 need longer runs, and Tests 9 and 10 will be in a separate script. We have to look at the trace plots in Test 7 ourselves.

8 Workflow

We will add this plan to our Git repository before we start coding, so the Git history shows that the plan came before the code. After that, all changes (including code, tests, and this plan) will go through pull requests, and the other person will review them in writing before merging.

If a test fails or is not useful, we will update the plan in a new commit and keep the old version in the history. The commit message and the report will explain what changed and why.

References

[1] D. Foreman-Mackey, D. W. Hogg, D. Lang and J. Goodman, "emcee: The MCMC Hammer," PASP **125**, 306 (2013).

[2] A. Vehtari, A. Gelman, D. Simpson, B. Carpenter and P.-C. Bürkner, "Rank-normalization, folding, and localization: an improved $\hat{R}$ for assessing convergence of MCMC," Bayesian Analysis **16**, 667 (2021).

[3] A. D. Sokal, *Monte Carlo Methods in Statistical Mechanics: Foundations and New Algorithms*, lecture notes (1996).